

compiled the *eval* in my paper into [IBM] 704 machine code, fixing bugs, and then advertised this as a LISP interpreter, which it certainly was. So at that point LISP had essentially the form that it has today...."

Suddenly, in a matter of weeks, I think, McCarthy found his theoretical exercise transformed into an actual programming language—and a more powerful one than he had intended. (Read the complete essay here: <http://www.paulgraham.com/icad.html>)

Now, despite how much I love LISP and think that all the pages of LFY should be dedicated to it, I also know that if I want to keep writing for LFY, I need to stick to the word limit. So, as interesting as the origin of LISP is, we should now move on to the real task at hand... (and for those who've lost the plot, it is *learning LISP*).

Defining local variables in LISP

To define a local variable, use the *let* command. A *let* expression has two parts; the first is a list of variable declarations, where we can declare one or more local variables—valid within the body of the *let*. The second is the body of the *let*, where we can use these variables. The expressions in the body are evaluated in order. Here's an example:

```
> (let ( (x 18)
        (y 13)
        (z 15))
    (+ x y z))
46
```

Here, I have defined three local variables *x*, *y* and *z*, and assigned them values of 18, 13 and 15, respectively. Each pair of local variable names and values assigned to them need to be surrounded by a set of parentheses. Also, the indentation, white-space, and line breaks are discretionary—and therefore, you can stack up the variables and their values in a *let* expression to resemble a table. This is why I have placed *y* directly underneath *x* and *z* directly below *y*. Figure 1 identifies the parts of the *let*.

Defining local functions in LISP

To define a local function, use the *flet* command. This, too, has two parts: (1) function declaration, and (2) the (*flet*) body, where the function may be called. The function declaration itself consists of a function name, the arguments to that function, and the function body, where we put the function's code. As with *let*, we can define one or more functions within the scope of the *flet*. But remember that these are local functions visible only in the body—they may not call one another, and so cannot be recursive. Here's an example:

```
> (flet ( (foo(n)
        (+ n 5))
        (bar(n)
        (+ (foo 2) n)))
  (bar 10))
```

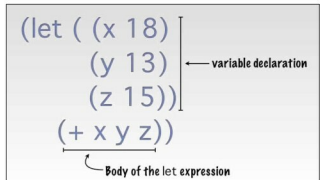


Figure 1: Let expression

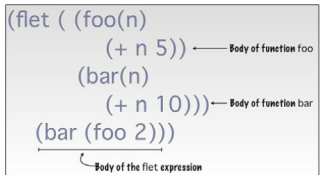


Figure 2: Flet expression

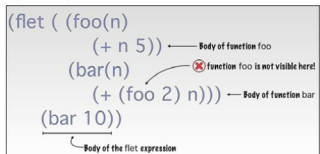


Figure 3: Error

```
(bar(n)
  (+ n 10))
(bar (foo 2)))
17
```

Here, I have declared two functions: *foo* and *bar*. In this code example, the body of the *flet* first calls *foo* with 2 as its argument, to return a value of 7; it then calls *bar* to add 10 to this value, leading to 17 as the final value of the whole *flet* expression. See Figure 2.

Let's see what happens when *bar* tries to call *foo* in its function body:

```
> (flet ( (foo(n)
        (+ n 5))
        (bar(n)
        (+ (foo 2) n)))
  (bar 10))
```